Міністерство освіти і науки України
Харківський національний університет радіоелектроніки
Кафедра програмної інженерії

Звіт
з лабораторної роботи №1
з предмету «Високорівневі мови програмування та фреймворки»

Виконав:
ст. гр. ПЗПІ 23-7
Варченко Микола Миколайович

Прийняв:
Саманцов О.О.

Харків 2026

1.1 Хід роботи

Рівень 1

1. Створіть Django-проект та додаток, який має модель "Стаття" з полями "Заголовок", "Текст" та "Дата публікації".

Встановлюємо Django.

```
$ pip3 install Django
Collecting Django
  Downloading django-6.0.3-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from Django)
  Downloading asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from Django)
  Downloading sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: tzdata in c:\users\humster\appdata\local\programs\python\python313\lib\site-packages (from Django) (2025.
2)
Downloading django-6.0.3-py3-none-any.whl (8.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.4/8.4 MB 9.2 MB/s eta 0:00:00
Downloading asgiref-3.11.1-py3-none-any.whl (24 kB)
Downloading sqlparse-0.5.5-py3-none-any.whl (46 kB)
Installing collected packages: sqlparse, asgiref, Django
Successfully installed Django-6.0.3 asgiref-3.11.1 sqlparse-0.5.5

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Рис.1.1 – Результат вставновлення фреймворку

Створюємо проект в бажаній папці.

```
C:\vs projjjects\vmptf_lb_1>django-admin startproject laba1
```

Рис.1.2 – Створення проєкту laba1

Переходимо в щойно створену папку та створюємо додаток.

```
C:\vs projjjects\vmptf_lb_1>cd laba1

C:\vs projjjects\vmptf_lb_1\laba1>python manage.py startapp articles
```

Рис.1.3 – Створення додатку articles

В файлі settings.py до списку INSTALLED_APPS додаємо ‘articles’:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'articles',
]
```

В файлі models.py створюємо модель:

```
from django.db import models

class Article(models.Model):
    title = models.CharField(max_length=255)
    text = models.TextField()
    publication_date = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

Рівень 2

1. Розширте модель "Стаття", додавши поле "Автор". За допомогою Django ORM, виведіть на екран інформацію про всі статті конкретного автора.

Додаємо поле author до моделі:

```
    author = models.ForeignKey('auth.User',
on_delete=models.CASCADE, null=True)
```

Виконуємо в терміналі, знаходячись на рівні файлу manage.py, команди:
python manage.py makemigrations
python manage.py migrate

```
C:\vs projjjects\vmptf_lb_1\laba1>python manage.py makemigrations
Migrations for 'articles':
  articles\migrations\0001_initial.py
    + Create model Article

C:\vs projjjects\vmptf_lb_1\laba1>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, articles, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying articles.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

Рис.1.4 – Результат виконання міграцій

Запускаємо інтерктивну оболонку.

```
C:\vs projjjects\vmptf_lb_1\laba1>python manage.py shell
13 objects imported automatically (use -v 2 for details).

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
```

Рис.1.5 – Створення додатку articles

Тепер в інтерактивну консоль пишемо наступний код, а саме створюємо записи та виводимо всі статті student'а:

```
from articles.models import Article
from django.contrib.auth.models import User

user = User.objects.create(username='student')

Article.objects.create(title='First Article', text='Some text',
author=user)
Article.objects.create(title='Second Article', text='Another text',
author=user)

articles = Article.objects.filter(author=user)

for article in articles:
    print(article.title, article.author.username)
```

Натискаємо Enter і бачимо наступне:

```
>>> from articles.models import Article
>>> from django.contrib.auth.models import User
>>>
>>> user = User.objects.create(username='student')
>>>
>>> Article.objects.create(title='First Article', text='Some text', author=user)
<Article: First Article>
>>> Article.objects.create(title='Second Article', text='Another text', author=user)
<Article: Second Article>
>>>
>>> articles = Article.objects.filter(author=user)
>>>
>>> for article in articles:
...     print(article.title, article.author.username)
...
First Article student
Second Article student
>>>
```

Рис.1.6 – Вивід всіх статей автора student

Для виходу пишемо exit().

```
>>> exit()
now exiting InteractiveConsole...

C:\vs projjjects\vmptf_lb_1\laba1>
```

Рис.1.7 – Вихід з інтерактивної оболонки

Рівень 3

1. Створіть шаблон Django для відображення списку статей та окремої статті. Застосуйте цей шаблон до веб-інтерфейсу.

В файлі views.py пишемо код для отримання та повернення всіх статей, та окремої:

```
from django.shortcuts import render, get_object_or_404
from .models import Article

def article_list(request):
    articles = Article.objects.all()
    context = {
        'articles': articles
    }
    return render(request, 'articles/article_list.html', context)

def article_detail(request, article_id):
    article = get_object_or_404(Article, id=article_id)
    context = {
        'article': article
    }
    return render(request, 'articles/article_detail.html', context)
```

Тепер зв’язуємо написані функції з конкретними веб адресами, за якими ці функції будуть викликані.

Створюємо файл urls.py на рівні файлу views.py з наступним контентом:

```
from django.urls import path
from . import views

urlpatterns = [
```

```
    path('', views.article_list, name='article_list'),

    path('<int:article_id>/', views.article_detail,
name='article_detail'),
]
```

В файлі urls.py в папці laba1 (на рівні settings.py) додаємо до списку urlpatterns:

```
path('articles/', include('articles.urls')),
```

Створюємо папку templates в articles. В ній папку articles. А в ній в свою чергу файли arctile_list.html та article_detail.html.

arctile_list.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Articles List</title>
</head>
<body>
    <h1>All Articles</h1>

    <ul>
        {% for article in articles %}
            <li>
                <a href="{% url 'article_detail' article.id %}">{{
article.title }}</a>
                <p>Author: {{ article.author.username }}</p>
                <p>Published: {{ article.publication_date }}</p>
            </li>
        {% endfor %}
    </ul>
</body>
</html>
```

article_detail.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>{{ article.title }}</title>
</head>
```

```
<body>
    <h1>{{ article.title }}</h1>
    <p><strong>Author:</strong> {{ article.author.username }}</p>
    <p><strong>Published:</strong> {{ article.publication_date
}}</p>

    <hr>

    <p>{{ article.text }}</p>

    <br>
    <a href="{% url 'article_list' %}">Back to articles list</a>
</body>
</html>
```

Запускаємо сервер:

python manage.py runserver

```
C:\vs projjjects\vmptf_lb_1\laba1>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
March 24, 2026 - 18:14:00
Django version 6.0.3, using settings 'laba1.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рис.1.8 – Сервер запущено

За шляхом http://127.0.0.1:8000/articles/ бачимо:

Рис.1.9 – Наповнення сторінки зі списком усіх статей

При переході на конкретну статтю (http://127.0.0.1:8000/articles/3/):

Рис.1.10 – Наповнення сторінки певної статті

Рівень 4

1. Додайте API-вид статей, який дозволяє витягувати дані статей у форматі JSON. Використовуйте Django REST Framework.

Встановлюємо бібліотеку Django REST Framework:

```
C:\vs projjects\vmptf_lb_1\laba1>pip install djangorestframework
Collecting djangorestframework
  Downloading djangorestframework-3.17.1-py3-none-any.whl.metadata (7.9 kB)
Requirement already satisfied: django>=4.2 in c:\users\humster\appdata\local\programs\python\python313\lib\site-packages (from djangorestframework) (6.0.3)
Requirement already satisfied: asgiref>=3.9.1 in c:\users\humster\appdata\local\programs\python\python313\lib\site-packages (from django>=4.2->djangorestframework) (3.11.1)
Requirement already satisfied: sqlparse>=0.5.0 in c:\users\humster\appdata\local\programs\python\python313\lib\site-packages (from django>=4.2->djangorestframework) (0.5.5)
Requirement already satisfied: tzdata in c:\users\humster\appdata\local\programs\python\python313\lib\site-packages (from django>=4.2->djangorestframework) (2025.2)
Downloading djangorestframework-3.17.1-py3-none-any.whl (898 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 898.8/898.8 kB 4.0 MB/s eta 0:00:00
Installing collected packages: djangorestframework
Successfully installed djangorestframework-3.17.1

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Рис.1.11 – Бібліотеку встановлено

В файлі settings.py до списку INSTALLED_APPS додаємо ‘rest_framework’:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'articles',
    'rest_framework',
]
```

Створюємо серіалізатор. В папці додатку articles створюємо файл serializers.py з наступним вмістом:

```
from rest_framework import serializers
from .models import Article

class ArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = ['id', 'title', 'text', 'publication_date',
'author']
```

В файлі views.py створюємо новий api-контролер імпортувавши наш серіалайзер:

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .serializers import ArticleSerializer

@api_view(['GET'])
def article_api_list(request):
    articles = Article.objects.all()
    serializer = ArticleSerializer(articles, many=True)
    return Response(serializer.data)
```

До файлу urls.py додаємо новий шлях:

```
    path('api/', views.article_api_list, name='article_api_list'),
```

Запускаємо сервер:

```
C:\vs projjjects\vmptf_lb_1\laba1>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
March 25, 2026 - 16:43:23
Django version 6.0.3, using settings 'laba1.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рис.1.12 – Сервер запущено

І за посиланням http://127.0.0.1:8000/articles/api/ бачимо наші статті у форматі JSON:

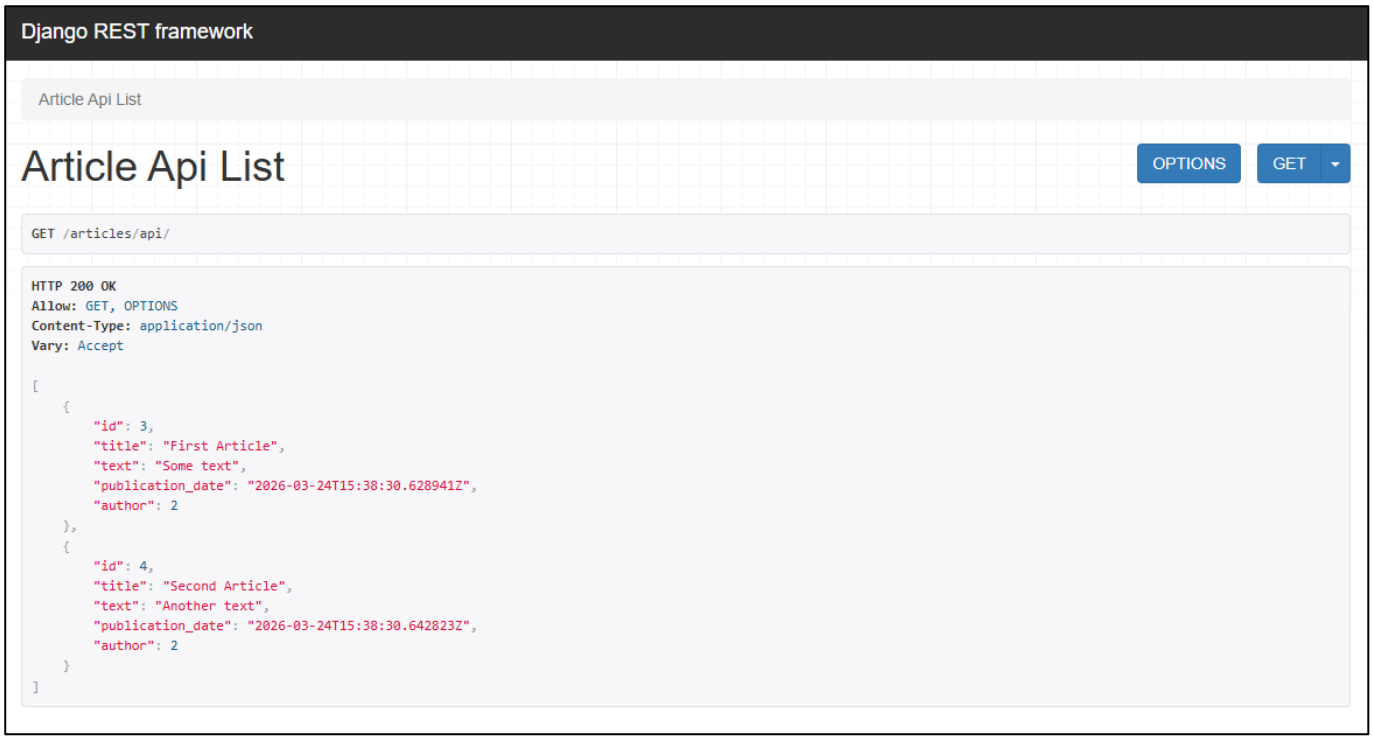

Рис.1.13 – Дані статей у JSON

1.2 Висновки

В ході виконання лабораторної роботи було розглянуто процес створення веб-додатку за допомогою фреймворку Django, зокрема створення проєкту та додатку, налаштування середовища та підключення компонентів.

Реалізовано модель даних із використанням ORM, виконано міграції та здійснено роботу з базою даних, включаючи створення і вибірку записів.

Опановано принципи побудови веб-інтерфейсу через представлення, маршрутизацію та шаблони для відображення списку об’єктів і детальної інформації, а також отримано базові навички створення API із використанням Django REST Framework для передачі даних у форматі JSON.